

aci_lib User Guide

ACI-CO processed-data toolkit

August 6, 2026

Abstract

This is a practical how-to for the contents of this repository: `src/aci_lib/` (a small Python package for reading already-computed ACI-CO pipeline outputs into pandas/xarray) and `notebooks/aci_lib_quickstart.ipynb` (a Google Colab notebook built on top of it). For *why* the underlying data looks the way it does – the ACI-CO methodology, the pipeline that produced it, which scripts wrote which files – see the companion `ARCHITECTURE.pdf` in this same repository instead. This document only covers using what is already built.

Contents

1	What’s in this repository	2
2	Setup	2
2.1	Requirements	2
2.2	Getting the code onto <code>sys.path</code>	2
3	Supplying the data	3
4	API reference	3
4.1	<code>set_data_dir(path)</code>	3
4.2	<code>get_data_dir()</code>	4
4.3	<code>list_datasets(base_dir=None, kind=None)</code>	4
4.4	<code>describe(name, base_dir=None)</code>	4
4.5	<code>load(name, base_dir=None)</code>	4
5	How dataset names are built	5
6	Common recipes	5
6.1	Plot a national time series	5
6.2	Pull one month out of a gridded series	5
6.3	Compare the same variable across regions	6
6.4	Only browse what is small/tabular first	6
7	Performance notes	6
8	Troubleshooting	6
9	Running the Colab notebook end to end	7

1 What's in this repository

Path	Contents
src/aci_lib/	The library itself: four small modules (Section 4). No third-party code, only pandas/xarray at call time.
notebooks/aci-lib-quickstart.ipynb	A ready-to-run Colab notebook: clones this repo, points aci_lib at your data, browses the catalog, loads two example datasets, plots one (Section 9).
ARCHITECTURE.tex / .pdf	The source pipeline's architecture: scientific target, data flow, known issues. Reference material, not a how-to.

What is not in this repository: the data itself. `data/processed/` from the source ACI-CO project is roughly 13 GB across 8,000+ files – too large for git – so it is never committed here. Every workflow below starts by pointing `aci_lib` at a copy of that directory that you supply (Section 3).

2 Setup

2.1 Requirements

`aci_lib` itself has no hard dependencies beyond the standard library at import time; individual functions import what they need lazily:

- `pandas` – always (used by `list_datasets()`, and by `load()` for CSV/XLSX).
- `openpyxl` – only if you load an `.xlsx` dataset.
- `xarray` and `dask` – only if you load a NetCDF dataset.
- A NetCDF backend for `xarray` (`netCDF4` or `h5netcdf`) – same condition.
- `matplotlib` – only if you plot; not imported by the library at all.

In Google Colab, all of these are preinstalled. Locally, install them with your usual environment manager if they are not already present (they are, in the source ACI-CO project's environment, since its own scripts use the same stack).

2.2 Getting the code onto `sys.path`

`aci_lib` is not published as a package – there is no `pip install`. You put `src/` on `sys.path` and import it directly.

In Colab:

```
!git clone --depth 1 https://github.com/mdgordillo/aca_aci_collab.git
import sys
sys.path.insert(0, "/content/aca_aci_collab/src")
import aci_lib
```

Locally, from a checkout of this repo:

```
import sys, os
sys.path.insert(0, os.path.abspath("src")) # or "../src" from notebooks/
import aci_lib
```

3 Supplying the data

`aci_lib` needs to know where a copy of `data/processed/` lives. It resolves this in the following order the first time it is needed:

1. An explicit `aci_lib.set_data_dir(path)` call.
2. The `ACI_DATA_DIR` environment variable.
3. Auto-detection: walking up from `src/aci_lib/` to the nearest directory containing a `.git` folder (the repo root), then checking whether `data/processed/` exists under it.

If none of these resolve to a real directory, every `aci_lib` call that needs data raises `FileNotFoundError` with a message pointing back to this section.

Method	When to use it	How
Local checkout of the <i>source</i> project	You have the ACI-CO pipeline repo cloned locally, with <code>data/processed/</code> already populated on disk	Nothing to do – auto-detection (step 3 above) finds it.
Mount Google Drive	Colab, and you have already uploaded <code>data/processed/</code> to your Drive	<code>drive.mount("/content/drive")</code> , then <code>aci_lib.set_data_dir("/content/drive/MyDrive/<path>/processed")</code>
Upload + unzip	Colab, one-off, no Drive setup wanted	<code>files.upload()</code> a zip, !unzip it, then <code>set_data_dir()</code> to the unzipped path
Environment variable	Any script/notebook where you'd rather configure via environment than code	<code>export ACI_DATA_DIR=/path/to/processed</code> (shell) or <code>os.environ["ACI_DATA_DIR"] = ...</code> before importing

All four are demonstrated in `notebooks/aci_lib_quickstart.ipynb`.

4 API reference

Four functions, all imported directly off the `aci_lib` package.

4.1 `set_data_dir(path)`

Points the library at a `data/processed`-like directory for the rest of the session. Raises `FileNotFoundError` immediately if `path` is not a directory (fails fast rather than deferring the error to the first `load()` call).

```
aci_lib.set_data_dir("/content/drive/MyDrive/aca_data/processed")
```

4.2 get_data_dir()

Returns whatever directory `aci_lib` is currently configured to read from (see the resolution order in Section 3). Useful to sanity-check setup before calling `list_datasets()`:

```
print(aci_lib.get_data_dir())
```

4.3 list_datasets(base_dir=None, kind=None)

Returns a `pandas.DataFrame`, one row per loadable dataset, columns `name`, `kind`, `n_files`, `description`. `base_dir` overrides the configured data directory for this call only (does not call `set_data_dir`). `kind` filters to one of "tabular", "grid", "grid_series", "image", "raw" (Section 5 explains what each means).

```
aci_lib.list_datasets() # everything
aci_lib.list_datasets(kind="tabular") # only CSV/XLSX datasets
aci_lib.list_datasets()[lambda d: d.name.str.contains("colombia")]
```

4.4 describe(name, base_dir=None)

Returns the underlying `Dataset` record for one catalog entry: its `name`, `kind`, the full list of file paths it is built from, and its `description`. Useful to see exactly which files a `grid_series` dataset will open before loading potentially gigabytes of NetCDF.

```
ds = aci_lib.describe("anomalias_colombia/anomalies_temperature")
print(ds.kind, ds.n_files)
print(ds.files[:3])
```

4.5 load(name, base_dir=None)

The main entry point. Returns different Python types depending on the dataset's `kind`:

Kind	Returns	Notes
tabular	<code>pandas.DataFrame</code>	<code>pd.read_csv</code> or <code>pd.read_excel</code> depending on the source file's extension.
grid	<code>xarray.Dataset</code>	A single NetCDF file, opened with <code>xr.open_dataset</code> .
grid_series	<code>xarray.Dataset</code>	Many NetCDF files (e.g. one per year-month) opened lazily with <code>xr.open_mfdataset</code> and concatenated along a <code>file</code> dimension, in chronological order. Backed by <code>dask</code> – see Section 7.
image	<code>str</code>	The file path (a <code>.png</code>); open it with <code>PIL.Image.open(...)</code> or <code>plt.imread(...)</code> yourself.
raw	<code>str</code>	The file path for anything else (<code>.txt</code> , <code>.ascii</code> , <code>.zip</code> , ...); read it yourself.

Raises `KeyError` if `name` is not in the catalog – the message points back to `list_datasets()`.

```
df = aci_lib.load("anomalias_colombia/anomalies_temperature_combined")
grid = aci_lib.load("anomalias_colombia/anomalies_temperature")
img_path = aci_lib.load("confusion_matrix")
```

5 How dataset names are built

Names mirror `data/processed/`'s directory structure, with the file extension stripped:

- A file directly under `data/processed/`, e.g. `ungrd_monthly.csv`, becomes `"ungrd_monthly"` (`kind="tabular"`).
- A file in a subdirectory, e.g. `anomalias_colombia/anomalies_temperature_combined.csv`, becomes `"anomalias_colombia/anomalies_temperature_combined"`.
- A *run* of NetCDF files that share a prefix and differ only by a trailing `_YYYY` or `_YYYY_MM`, e.g. `anomalies_temperature_1961_1.nc` through `_2024_12.nc` (768 files), collapses to one entry named after the shared prefix – `"anomalias_colombia/anomalies_temperature"` – with `kind="grid-series"`.
- A single file that happens to match that same `_YYYY[_MM]` pattern but has no siblings (e.g. a baseline file literally named `aci_temp_1961_1990.nc`) is *not* treated as a series – it keeps its full name and `kind="grid"`.

When in doubt, do not guess a name – call `aci_lib.list_datasets()` and filter/search the `name` column.

6 Common recipes

6.1 Plot a national time series

```
import matplotlib.pyplot as plt

df = aci_lib.load("anomalias_colombia/anomalies_temperature_combined")
df = df.sort_values(["year", "month"])
plt.plot(range(len(df)), df["t_90"])
plt.title("National T90 anomaly, monthly")
plt.show()
```

6.2 Pull one month out of a gridded series

```
grid = aci_lib.load("anomalias_colombia/anomalies_temperature")
first_month = grid.isel(file=0) # already chronologically first
first_month["t_90"].plot()
```

6.3 Compare the same variable across regions

```
regions = ["antioquia", "cali", "colombia", "cundinamarca_bogota", "valle_cauca"]
series = {
    r: aci_lib.load(f"anomalias_{r}/anomalies_temperature_combined")
    for r in regions
}
```

6.4 Only browse what is small/tabular first

Gridded datasets can be gigabytes; when exploring interactively, start with `kind="tabular"` to avoid triggering large lazy loads by accident:

```
aci_lib.list_datasets(kind="tabular")
```

7 Performance notes

- `grid_series` datasets are opened with dask chunking (one chunk per source file) and are *lazy*: `aci_lib.load(...)` itself does not read the underlying data into memory. Values are only computed when you call `.values`, `.compute()`, or `plot`.
- Subset (`.isel/.sel`) before triggering computation, not after – e.g. `grid.isel(file=0)["t_90"].plot()` reads one file’s worth of data; `grid["t_90"].values` reads all of them.
- `list_datasets()` and `describe()` only walk the directory tree and match filenames – they never open a data file, so they are cheap to call repeatedly.

8 Troubleshooting

Symptom	Fix
<code>FileNotFoundError: Could not find data/processed/</code> <code>KeyError: No dataset named ...</code>	No data source configured yet – follow Section 3.
<code>ModuleNotFoundError: No module named 'xarray' (or dask, netCDF4)</code> <code>load()</code> on a <code>grid_series</code> is slow or uses a lot of memory Colab session restarted / variables gone	The name does not match the catalog exactly (case-sensitive, no leading/trailing <code>.csv</code>). Run <code>aci_lib.list_datasets()</code> and search/filter its <code>name</code> column instead of guessing. Only triggered when loading a NetCDF dataset. Install the missing package – not needed at all if you only load tabular datasets. You likely triggered a full compute on an unsubsetted dataset – see Section 7. <code>sys.path</code> , <code>aci_lib</code> ’s configured data directory, and any Drive mount are session state – re-run the setup cells at the top of the notebook.

9 Running the Colab notebook end to end

`notebooks/aci_lib_quickstart.ipynb` is organized to be run top to bottom:

1. **Colab detection** – a small cell sets `IN_COLAB` so the rest of the notebook behaves correctly whether it is running in Colab or from a local checkout.
2. **Get the code** – clones this repo (Colab) or adds the sibling `../src` to `sys.path` (local).
3. **Supply the data** – Drive-mount cell is active by default; the upload/unzip alternative is present but commented out. Edit the `DATA_DIR` path to match where you put your data.
4. **Browse the catalog** – calls `list_datasets()` and shows the first 20 rows.
5. **Load examples** – one tabular dataset, one gridded series, and a plot of the first month in that series.
6. **Reference** – pointers to `describe()` and to `ARCHITECTURE.pdf` for pipeline-level context.

Open it directly in Colab via the badge at the top of the notebook, or at: https://colab.research.google.com/github/mdgordillo/aca_aci_collab/blob/main/notebooks/aci_lib_quickstart.ipynb